

Project 1: Linear System

Report

Team member

Doan Thi Minh Ngoc - 104240572

Vo Khanh Tran - 10423177

Duong Huynh Anh Minh - 104240347

1. Project Overview

This project is split into two parts. Part A asks us to build a small numerical library from scratch - a Vector class, a Matrix class, and a LinearSystem class that can solve the equation $Ax = b$. Part B then uses that library to train a linear regression model on a real CPU performance dataset from UCI.

The overall flow is: build the math tools first, then apply them to a real problem. For the regression in Part B, the model is:

$$PRP = x_1 \cdot MYCT + x_2 \cdot MMIN + x_3 \cdot MMAX + x_4 \cdot CACH + x_5 \cdot CHMIN + x_6 \cdot CHMAX$$

To find the six parameters x_1 through x_6 , we set up an overdetermined system (more equations than unknowns) and reduce it to a solvable 6×6 system using the normal equations $(A^T A)x = A^T b$, then call our own `LinearSystem::Solve()` to finish it.

2. Part A: Building the Library

2.1 The Vector Class

The Vector class stores a dynamic array of double values using a raw pointer `mData`, and tracks the size in `mSize`. Because we use raw heap memory, we had to write three special functions ourselves: a constructor that allocates the array, a copy constructor that deep-copies it, and a destructor that frees it:

```
Vector(int size);                // allocates mData on the heap
Vector(const Vector& other);      // deep copy, not just copying
                                // the pointer
~Vector();                       // frees the memory
```

We overloaded `operator()` with 1-based indexing so `v(1)` is the first element, matching the math convention. We also overloaded `operator[]` with 0-based indexing and a bounds check via `assert`. Standard arithmetic operators (`+`, `-`, `*`) were overloaded, and `Dot()` and `Norm()` were added as utility methods.

One thing that needed care was the assignment operator - we check for self-assignment (if (this == &other)) and handle the case where sizes differ before reallocating memory.

2.2 The Matrix Class

The Matrix class follows the same ownership pattern but for a 2D array. The internal storage is `double** mData` - a pointer to an array of row pointers. Memory allocation and deallocation are split into private helper methods `Allocate()` and `Deallocate()` to avoid repeating that logic across the constructor, copy constructor, and assignment operator.

Key features implemented:

- 1-based indexing via `operator()(int i, int j)` - so `A(1,1)` maps to `mData[0][0]`
- Arithmetic operators for matrix-matrix, matrix-vector, and matrix-scalar multiplication
- Transpose - a new matrix built by swapping rows and columns
- Determinant - computed via LU decomposition with partial pivoting for numerical stability
- Inverse - computed via Gauss-Jordan elimination on the augmented matrix $[A | I]$
- Pseudo-inverse (Moore-Penrose) - uses $(A^T A)^{-1} A^T$ for overdetermined systems and $A^T (A A^T)^{-1}$ for underdetermined ones

Both the determinant and inverse use partial pivoting - swapping rows to put the largest value at the pivot position - to avoid dividing by very small numbers, which would cause numerical instability.

2.3 The LinearSystem Class

LinearSystem brings Vector and Matrix together. It holds a pointer to a matrix (`Matrix* mpA`) and a pointer to a right-hand-side vector (`Vector* mpb`), with the system size stored in `mSize`.

We made the default constructor and copy constructor private:

```
LinearSystem(const Matrix& A, const Vector& b);
```

The constructor takes both by const reference and stores deep copies on the heap. The destructor does `delete mpA` and `delete mpb`.

The `Solve()` method implements Gaussian elimination with partial pivoting. It works on copies of `mpA` and `mpb` so the originals are preserved, eliminates entries below each pivot column, then does back-substitution to find `x`. If any pivot falls below `1e-12`, it throws a `std::runtime_error` to signal a singular matrix.

2.4 The PosSymLinSystem Class (Derived Class)

PosSymLinSystem inherits from LinearSystem and overrides `Solve()` to use the Conjugate Gradient (CG) method instead of Gaussian elimination. CG is better suited for large symmetric positive definite systems because it only does repeated vector operations rather than building and eliminating a full matrix.

Because PosSymLinSystem needs access to mpA, mpb, and mSize from its parent, those members are declared protected in LinearSystem instead of private - this is the standard way to share data with derived classes without making it fully public.

The constructor calls the parent constructor via the initializer list:

```
PosSymLinSystem(const Matrix& A, const Vector& b) :  
    LinearSystem(A, b) { ... }
```

It also calls IsSymmetric() to validate the input - checking that $A(i,j)$ equals $A(j,i)$ within a tolerance of $1e-10$. Testing for positive definiteness would require checking all eigenvalues, which is much harder, so we skip it.

The CG algorithm maintains three vectors: the current solution x , the residual $r = b - Ax$, and the search direction p . Each iteration computes a step size α , updates x and r , then adjusts the direction using β . It stops when the residual norm drops below $1e-10$ or after 10,000 iterations.

Solve() is declared virtual in the base class, so calling it through a LinearSystem pointer that actually points to a PosSymLinSystem object will correctly dispatch to the CG version at runtime — this is runtime polymorphism. The virtual destructor on LinearSystem is also important here, to ensure the derived class destructor is called correctly when deleting through a base pointer.

2.5 Over- and Under-determined Systems

For non-square systems where a unique solution does not exist, we use the pseudo-inverse already built into the Matrix class:

- Overdetermined (more rows than columns): $x = (A^T A)^{-1} A^T b$ - least-squares solution minimizing $\|Ax - b\|$
- Underdetermined (more columns than rows): $x = A^T (A A^T)^{-1} b$ - minimum-norm solution

3. Part B: Linear Regression on CPU Dataset

3.1 Dataset

We use the UCI Computer Hardware dataset with 209 records. Each record has 10 fields: vendor name, model name, six numeric features (MYCT, MMIN, MMAX, CACH, CHMIN, CHMAX), a published performance score (PRP), and an estimated one (ERP). We skip the first two name fields and the ERP column, treating PRP as the target to predict.

Loading is handled in LinearRegression::LoadData(). It opens the file, reads line by line, splits each line by comma, and parses the six feature columns and PRP using std::stod. Lines with fewer than 10 columns or unparseable values are silently skipped.

3.2 Training

After loading, records are shuffled with a fixed random seed (for reproducibility) and split 80/20 into training (~167 records) and test (~42 records) sets.

We then build:

- Matrix A of size $(m \times 6)$, where each row holds one record's six feature values

- Vector b of size m , where each entry is that record's PRP value

Since $m \approx 167$ and we only have 6 unknowns, this is a heavily overdetermined system. We reduce it using the normal equations:

$$(A^T A) x = A^T b$$

This produces a 6×6 system that our existing `LinearSystem::Solve()` can handle directly. We add a small regularization term ($\lambda = 1e-6$) to the diagonal of $A^T A$ to keep it non-singular:

```
for (int i = 1; i <= n; ++i)
    AtA(i, i) += lambda; // small value to avoid singular
matrix problem
LinearSystem sys(AtA, Atb); return sys.Solve();
```

3.3 Evaluation

After training, we predict PRP for every record in the test set and compute the Root Mean Square Error (RMSE):

$$RMSE = \sqrt{(\sum(\text{predicted} - \text{actual})^2 / n)}$$

We print the six learned parameters, the training RMSE, the test RMSE, and a side-by-side table of five sample predictions vs actual values.

4. What We Asked AI For Help With

AI was used mainly as a checking tool while we were building and cleaning up the project. The main code structure still follows the project requirement: Part A builds the math classes, and Part B reuses those classes for linear regression.

One useful change was in `LinearRegression::Train()`. The first version repeated Gaussian elimination inside the regression file. After reviewing it, we changed that part to create a `LinearSystem` object and call `Solve()`. This made Part B depend on the Part A solver.

We also used AI to understand a few C++ points, especially why `LinearSystem` members such as `mpA`, `mpb`, and `mSize` should be protected for the derived class, and why `Solve()` should be virtual when `PosSymLinSystem` overrides it.

For the regression part, AI also helped us check the singular-matrix issue in $A^T A$. We added a small `lambda` value to the diagonal before solving, so the system is more stable when the dataset columns are highly related.

Overall, AI support was used for review, debugging ideas, and explanation. The final code was still tested and adjusted in our own project files.

5. Result

```
=====
PART B: Linear Regression on CPU Performance Dataset
=====
```

```
Loaded 209 records from data\machine.data
```

```
Training set size: 167
```

```
Testing set size: 42
```

```
Learned parameters:
```

```
x1 (MYCT) = -0.03215
```

```
x2 (MMIN) = 0.015758
```

```
x3 (MMAX) = 0.00412275
```

```
x4 (CACH) = 0.536555
```

```
x5 (CHMIN) = -0.94464
```

```
x6 (CHMAX) = 1.47916
```

```
Train RMSE: 70.2842
```

```
Test RMSE: 43.7071
```

```
Sample predictions (first 5 test records):
```

PRP_actual	PRP_predicted
489	449.875
45	55.5647
92	108.334
46	72.703
63	123.146

The training RMSE was 70.2842, while the testing RMSE was 43.7071. Since the test RMSE is lower than the training RMSE in this run, the model did not show an obvious overfitting problem. However, the error is still quite noticeable, which makes sense because the model is only a simple linear regression model with six input features. CPU performance may depend on more complex relationships than this simple formula can fully capture.

6. References

- GeeksForGeeks – Gaussian Elimination:
<https://www.geeksforgeeks.org/gaussian-elimination/>
- GeeksForGeeks – Conjugate Gradient Method:
<https://www.geeksforgeeks.org/deep-learning/second-order-optimization-methods/>
- GeeksForGeeks – Overloading Operators in C++:
<https://www.geeksforgeeks.org/operator-overloading-cpp/>
- UCI Machine Learning Repository – Computer Hardware Dataset:
<https://archive.ics.uci.edu/ml/datasets/Computer+Hardware>
- AI assistant - used for: (1) code review; (2) debugging singular matrix error caused by missing regularization; (3) clarifying protected vs private in the context of inheritance